MongoDB — Zero to Master

22 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (6 topics)

01 Introduction to NoSQL and MongoDB

02 Installation, Atlas and mongosh

03 Databases, Collections and Documents

04 Insert Documents

05 Find and Query Operators

06 Update and Delete

Intermediate (6 topics)

07 Data Modeling: Embedding vs Referencing

08 Indexes

09 Aggregation Pipeline

10 Schema Validation

11 Using MongoDB from Node.js

12 Array and Embedded Document Queries

Advanced (6 topics)

13 Transactions

14 Replication and Replica Sets

15 Sharding

16 Security

17 Performance Tuning

18 Change Streams and Time Series

Master (4 topics)

19	Atlas Search and Vector Search
20	Backup, Restore and Monitoring
21	Production Best Practices
22	Real Project: Blog / E-commerce Backend

Beginner Level

01. Introduction to NoSQL and MongoDB

MongoDB is a document database storing flexible JSON-like (BSON) documents in collections instead of rows in tables.

```
mongosh
show dbs
```

02. Installation, Atlas and mongosh

Install MongoDB Community or use free MongoDB Atlas in the cloud; connect with `mongosh` or Compass.

```
mongosh "mongodb+srv://cluster.example.mongodb.net/test"
```

03. Databases, Collections and Documents

A database holds collections; a collection holds documents with an `_id` primary key.

```
use shop
db.createCollection('products')
```

04. Insert Documents

`insertOne` and `insertMany`; documents may have nested objects and arrays.

```
db.products.insertOne({ name: 'Laptop', price: 55000, tags: ['tech'] })
```

05. Find and Query Operators

`find`, filters with `$gt`, `$lt`, `$in`, `$and`, `$or`, `$regex`; projection, sort, limit.

```
db.products.find({ price: { $gt: 1000 } }, { name: 1 }).sort({ price: -1 }).limit(5)
```

06. Update and Delete

`updateOne`, `updateMany` with `$set`, `$inc`, `$push`, `$pull`; `deleteOne`, `deleteMany`.

```
db.products.updateOne({ name: 'Laptop' }, { $inc: { price: -500 } })
```

Intermediate Level

07. Data Modeling: Embedding vs Referencing

Embed data that is read together; reference data that is large, shared or grows without bound.

```
{ _id: 1, name: 'Arun', addresses: [{ city: 'Chennai' }] }
```

08. Indexes

Single, compound, text, TTL and unique indexes; use `explain()` to verify.

```
db.users.createIndex({ email: 1 }, { unique: true })
```

09. Aggregation Pipeline

Process data in stages: `$match`, `$group`, `$project`, `$sort`, `$lookup`, `$unwind`.

```
db.orders.aggregate([
  { $match: { status: 'paid' } },
  { $group: { _id: '$city', total: { $sum: '$amount' } } }
])
```

10. Schema Validation

Enforce structure with JSON Schema validators on collections.

```
db.createCollection('users', { validator: { $jsonSchema: { required: ['email'] } } })
```

11. Using MongoDB from Node.js

Official driver or Mongoose ODM with schemas and models.

```
const User = mongoose.model('User', new Schema({ name: String }));
await User.create({ name: 'Priya' });
```

12. Array and Embedded Document Queries

`$elemMatch`, dot notation, array update operators and positional operators.

```
db.users.find({ 'addresses.city': 'Chennai' })
```

Advanced Level

13. Transactions

Multi-document ACID transactions on replica sets and sharded clusters.

```
const s = client.startSession();
await s.withTransaction(async () => { /* ops with { session: s } */ });
```

14. Replication and Replica Sets

Primary and secondaries provide high availability and automatic failover; read preferences and write concerns.

```
rs.status()
```

15. Sharding

Horizontal scaling by distributing data across shards using a shard key; choose a key with high cardinality.

```
sh.shardCollection('shop.orders', { customerId: 'hashed' })
```

16. Security

Enable authentication, role-based access control, TLS, IP allow lists, field-level encryption.

```
db.createUser({ user: 'app', pwd: '...', roles: [{ role: 'readWrite', db: 'shop' }] })
```

17. Performance Tuning

Right indexes, covered queries, projections, avoiding unbounded arrays, profiler and `explain('executionStats')`.

```
db.orders.find({ status: 'paid' }).explain('executionStats')
```

18. Change Streams and Time Series

React to data changes in real time; store time-stamped metrics efficiently.

```
db.orders.watch().on('change', c => console.log(c));
```

Master Level

19. Atlas Search and Vector Search

Full-text and semantic search directly in MongoDB Atlas for modern AI apps.

```
{ $search: { text: { query: 'laptop', path: 'name' } } }
```

20. Backup, Restore and Monitoring

`mongodump` / `mongorestore`, Atlas backups, monitoring, alerting and capacity planning.

```
mongodump --uri "$URI" --out backup/
```

21. Production Best Practices

Connection pooling, schema versioning, indexes review, data lifecycle (TTL, archiving), least privilege.

22. Real Project: Blog / E-commerce Backend

Model users, products and orders; build APIs with Express + Mongoose; add indexes, aggregation reports and Atlas deployment.

Interview Question Bank

Q1. SQL vs NoSQL?
SQL uses fixed relational schemas and joins; NoSQL (like MongoDB) uses flexible documents and scales horizontally.

Q2. What is BSON?
Binary JSON format with extra types (dates, ObjectId) used by MongoDB.

Q3. Embedding vs referencing?
Embed for data read together and bounded size; reference for large, shared or unbounded data.

Q4. What is the aggregation pipeline?
A sequence of stages that transform documents into aggregated results.

Q5. What is a replica set?
A group of mongod instances with one primary and secondaries for high availability.

Q6. What is sharding?
Partitioning data across servers using a shard key for horizontal scaling.

Q7. What is `_id`?
The mandatory unique primary key of every document, ObjectId by default.

Q8. How do indexes work in MongoDB?
B-tree structures that avoid collection scans; compound index order matters.

Q9. Does MongoDB support transactions?
Yes, multi-document ACID transactions since 4.0 (replica sets) and 4.2 (sharded).

Q10. What is Mongoose?
An ODM for Node.js providing schemas, validation and models.

Capstone Project Ideas

1. Blog backend
2. E-commerce catalogue with aggregation reports
3. Chat app storing messages
4. IoT time-series logger
5. Task manager API with Mongoose